



Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

ITEC 204 – Data Structures and Algorithms

Name: SUMARIA, FRANCHESKA ANNE A.	Date: 09/24/2026
Section: BSIT – 2A	Instructor: MS. MAXYNE IGNACIO

- I. **Objective:** Write a short explanation of what your program is designed to do. (Minimum of 3 sentences)

Answer: The program is designed to manage IT automation incident tickets using a Singly Linked List. It allows the user to add, display, search, remove resolved, and count active incident tickets. Each incident is stored as a node containing the Incident ID, bot name, description, and a link to the next ticket.

- II. **Program Operations:** Briefly explain how your program performs each operation. (Insert Specific Code Line and Your Explanation)

Add Incident:

CODE LINE:

```
new_node = Ticket(incident_id, bot, description)
while current.next:
    current = current.next
current.next = new_node
self.size += 1
```

EXPLANATION:

This program is an IT Incident Ticket Manager. It helps manage tickets made by automation bots. Each ticket has an Incident ID, the bot name, and a short description of the problem. The program lets the user add, view, search, remove, and count tickets using a menu. It starts with 10 sample tickets and stores them using a singly linked list.

Display Incidents:

CODE LINE:

```
while current:
    print(f'{current.incident_id:<14} {current.bot:<20} {current.description}')
    current = current.next
```



Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

EXPLANATION:

The program starts at the first ticket and checks each ticket one by one. It displays the Incident ID, Bot, and Short Description of each ticket. It stops when there are no more tickets to check. If there are no tickets, it displays “No active incident tickets.”

Search Incident:

CODE LINE:

```
if current.incident_id.upper() == incident_id.upper():  
    return current
```

EXPLANATION:

The program starts at the first ticket and checks each ticket one by one. It compares the Incident ID with the ID entered by the user, without considering uppercase or lowercase letters. If a match is found, the ticket details are shown. If no match is found, the program displays “Ticket not found.”

Remove Resolved Incident:

CODE LINE:

```
if previous is None:  
    self.head = current.next  
else:  
    previous.next = current.next  
self.size -= 1
```

EXPLANATION:

The program checks each ticket while keeping track of the previous ticket. When it finds the matching Incident ID, it removes that ticket from the list. If it is the first ticket, the head moves to the next ticket. Otherwise, the previous ticket is connected to the next ticket. The size is then reduced by 1. If no match is found, the program says that the ticket was not found.

Count Active Incidents:

CODE LINE:

```
return self.size
```

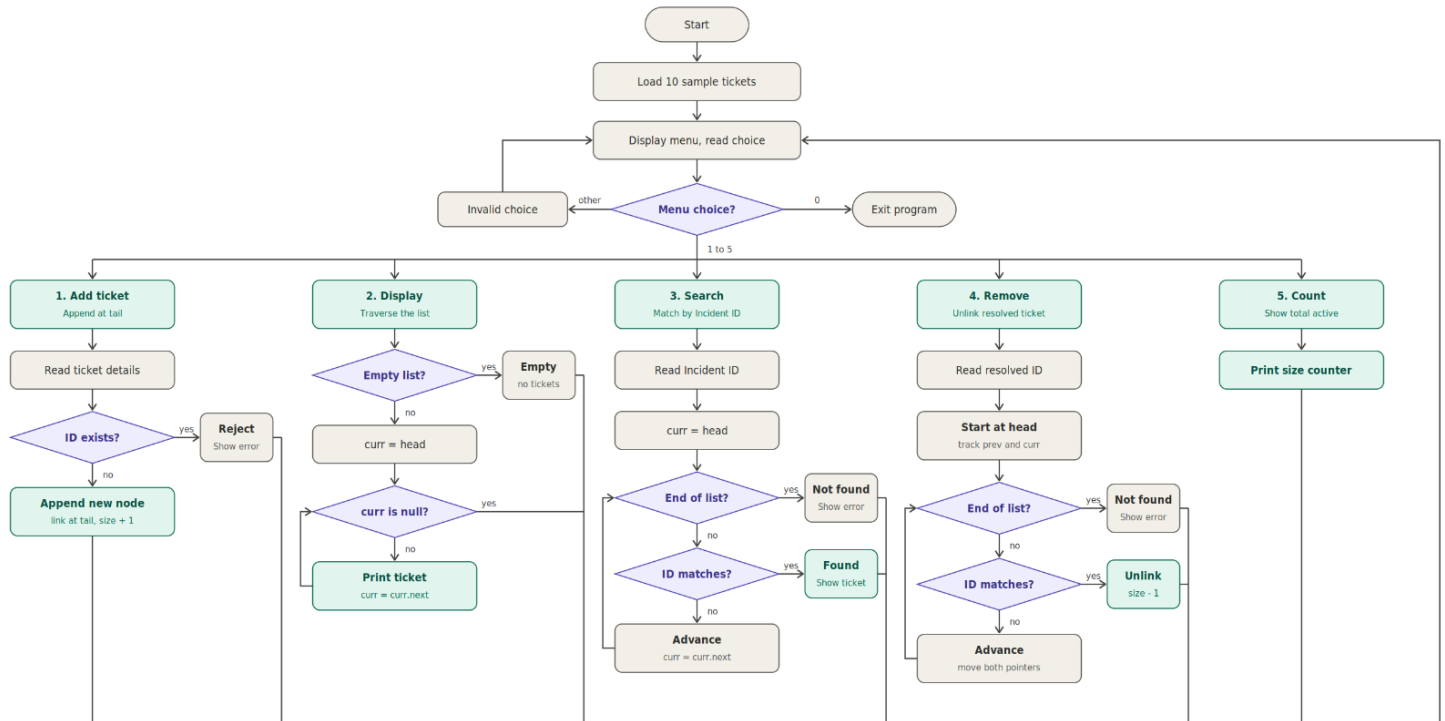
EXPLANATION:

The program uses a size counter to keep track of the number of tickets. It increases when a ticket is added and decreases when a ticket is removed. To count the tickets, the program simply returns the current value of size.



College of Computer Studies

III. **Flowchart:** Create a flowchart showing the overall process and logic of your program.



IV. **Conclusion:** Briefly explain what you learned from the activity and how the linear data structure was applied.

Through this activity, I learned how to bridge theoretical computer science theories with operational IT problem-solving requirements. A linear data structure, specifically a dynamic list array was effectively applied to hold incident structures sequentially in memory, enabling safe indexed traversal, element shifting, and lookup routines. This project reinforced my understanding that choosing the correct structural system ensures data integrity, logical workflow sequencing, and predictable performance behaviors when managing software state records.
